

Лабораторная работа № 3

Растеризация отрезков прямых

Цель работы – изучение алгоритмов растеризации отрезков, создание программы для визуализации работы алгоритмов.

Краткие теоретические сведения

Дисплей накладывает технические ограничения на представление графической информации. Изображение представляется в виде точек, и каждой точке соответствует вполне определённый атрибут – цвет и интенсивность. Таким образом, графические (а точнее – дисплейные) примитивы хранятся в памяти в виде совокупности образующих их точек, называемых *пикселями*.

В процессе вывода изображение (кадр) помещается в битовую карту (видеопамять), которая организована по линейному принципу и расположена в адресном пространстве центрального процессора. Но прежде чем будет сформирован растровый образ в видеопамяти, файл должен быть преобразован в соответствующий формат вывода. Чтобы преобразовать двумерный точечный образ, необходимо проанализировать и преобразовать значение каждой точки, что резко снижает производительность видеоподсистемы и системы в целом. Когда изображение может быть представлено в виде совокупности графических примитивов, т. е. в векторном формате, то процесс преобразования сводится к растеризации отдельных отрезков, окружностей, областей заливки и т. д., следовательно, количество анализируемых и синтезируемых точек намного меньше, чем при обработке растровых изображений. К тому же файлы векторного формата в среднем на порядок меньше по объёму растровых файлов, поскольку последние хранят описание каждого пикселя и объём резко увеличивается с увеличением разрешения.

Базовые алгоритмы предназначены для реализации (растеризации) графических примитивов, таких как отрезки, окружности, эллипсы и т. д. (Графические примитивы, описанные на языке высокого уровня, – это векторная форма изображения.) Кроме того, в эту группу попадают такие операции машинной графики, как закраска сплошных областей и отсечение отрезков и многоугольников.

Необходимость разработки специальных алгоритмов построения геометрических объектов объясняется тем, что сами эти объекты имеют непрерывную (аналоговую) природу, а изображение строится, как правило, на экране растрового дисплея, т. е. получаемое изображение носит дискретный характер. Это означает, например, что нельзя построить непосредственно отрезок, соединяющий две заданные точки экрана. Решение поставленной задачи очевидно лишь для трёх типов отрезков: горизонтальных, вертикальных и под углом 45° (рис. 1). Процесс нахождения пикселей, наилучшим образом аппроксимирующих заданный отрезок, называется разложением отрезка в растр.

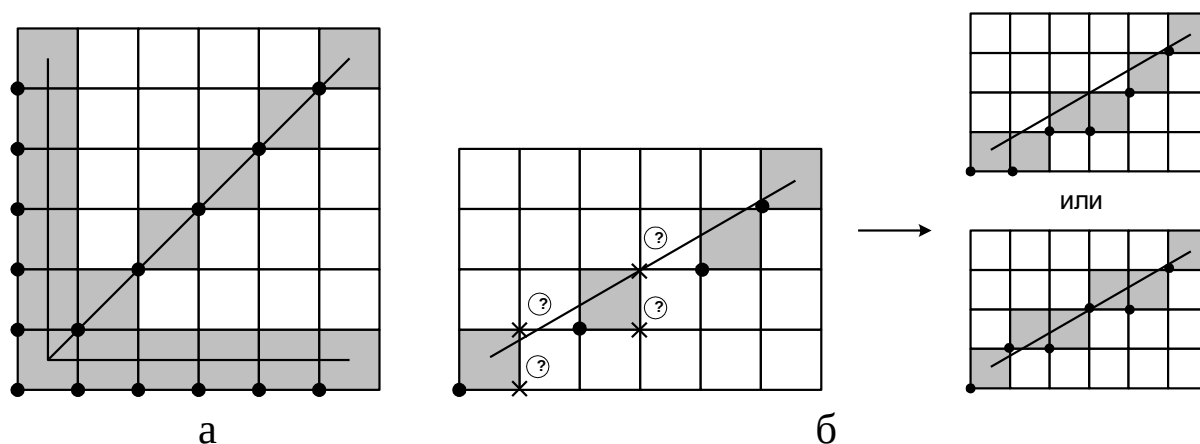


Рис. 1. Разложение отрезка в растр: а – тривиальный случай; б – поиск наилучшего приближения произвольного отрезка пикселями

Требования к алгоритмам растеризации отрезков.

Во-первых, отрезки должны выглядеть прямыми, начинаться и заканчиваться в заданных точках. Во-вторых, яркость вдоль отрезка должна быть постоянной и не зависеть от длины и наклона. В-третьих, алгоритмы должны работать быстро.

Первое требование в силу дискретной природы растрового дисплея выполнено всегда быть не может. Можно лишь добиться того, что визуально (человеческим глазом) отрезок будет восприниматься прямым. Решение этой задачи может достигаться путём увеличения разрешающей способности экрана дисплея и применения методов устранения ступенчатости.

Второму требованию удовлетворяют также только горизонтальные, вертикальные и наклонённые под углом в 45° отрезки. Однако вертикальные и горизонтальные отрезки по сравнению с отрезками,

расположенными под 45° , будут выглядеть ярче, так как расстояние между соседними пикселями у них меньше, чем у наклонных отрезков. Обеспечение постоянной яркости вдоль отрезка требует высвечивания очередного пикселя яркостью, зависящей от расстояния между пикселями, вычисление которого производится с использованием операций извлечения квадратного корня и умножения. Использование этих операций существенно замедляет работу алгоритма, поэтому второе требование остаётся, как правило, невыполненным.

Удовлетворение третьего требования достигается путём сведения к минимуму вычислительных операций, использования операций над целочисленными данными, а также реализацией алгоритмов на аппаратном или микропрограммном уровне.

Цифровой дифференциальный анализатор.

Многие алгоритмы вычерчивания отрезков и кривых используют пошаговый принцип, суть которого состоит в том, что результат вычисления координат высвечиваемого пикселя зависит от результатов, полученных на предыдущем шаге. Алгоритм вычерчивания отрезков по методу цифрового дифференциального анализатора, кроме этого, основан также на достаточно общем принципе, известном в математике: изучение какого-либо явления на основе дифференциального уравнения или системы таких уравнений, описывающей это явление.

Поскольку прямая линия на плоскости описывается уравнением вида

$$ax + by + c = 0,$$

где a , b , c – коэффициенты этого уравнения, производная dy/dx является постоянной.

Заменив дифференциалы конечными разностями, получим

$$\frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1},$$

где x_1, y_1 и x_2, y_2 – координаты начальной и конечной точек отрезка.

Ордината очередного пикселя y_{i+1} может быть вычислена по известной ординате предыдущего пикселя y_i следующим образом:

$$y_{i+1} = y_i + \Delta y.$$

Подставляя Δy , получим

$$y_{i+1} = y_i + \frac{y_2 - y_1}{x_2 - x_1} \Delta x.$$

Остаётся определить величину приращения Δx . Как правило, большее из приращений (Δx или Δy) выбирается в качестве единицы раstra, а приращение вдоль другой координатной оси подлежит определению. Если же поступить по-другому (меньшее из приращений взять равным единице), то отрезок на экране может получиться «дырявым», т. е. состоящим из отдельных точек, не расположенных вплотную друг к другу.

Алгоритм разложения отрезка в растр по методу ЦДА может быть записан следующим образом:

1. Ввод исходных данных: x_1, y_1, x_2, y_2 .
2. Проверка вырожденности отрезка. Если отрезок вырожден, то вывод точки и переход к п.7.
3. Вычисление $L:=|x_2-x_1|$, если $|x_2-x_1|\geq|y_2-y_1|$ или $L:=|y_2-y_1|$, если $|y_2-y_1|>|x_2-x_1|$.
4. Вычисление $dx:=(x_2-x_1)/L$, $dy:=(y_2-y_1)/L$.
5. Присвоение координатам текущей точки начальных значений: $x:=x_1+0.5*\text{sign}(dx)$, $y:=y_1+0.5*\text{sign}(dy)$, где sign – функция, возвращающая -1, 0, 1 для отрицательного, нулевого и положительного аргумента соответственно, использование этой функции делает алгоритм пригодным для всех квадрантов.
6. Цикл от $i:=1$ до $i=L+1$ с шагом 1:
 - вывод точки с текущими координатами ($\text{floor}(x), \text{floor}(y)$), где floor – функция округления до ближайшего меньшего целого;
 - вычисление координат следующей точки: $x:=x+dx$, $y:=y+dy$.
7. Конец.

Вещественный алгоритм Брезенхема

Работа алгоритма Брезенхема основывается на использовании понятия ошибка. Ошибкой здесь называется расстояние между действительным положением отрезка и ближайшим пикселем сетки раstra, который аппроксимирует отрезок на очередном шаге.

На каждом шаге вычисляется величина ошибки и в зависимости от полученного значения выбирается пиксель, ближе расположенный к идеальному отрезку. Если отрезок (рис.2) начинается в пикселе (0,0) и пересекает прямую $x=1$ между пикселями (1,0) и (1,1), то выбор одного из этих пикселей для аппроксимации отрезка зависит от величины тан-

генса угла наклона отрезка $\Delta y/\Delta x$. Поскольку при реализации алгоритма на ЭВМ удобнее анализировать не само значение ошибки, а её знак, то истинное значение ошибки смещается на $-0,5$.

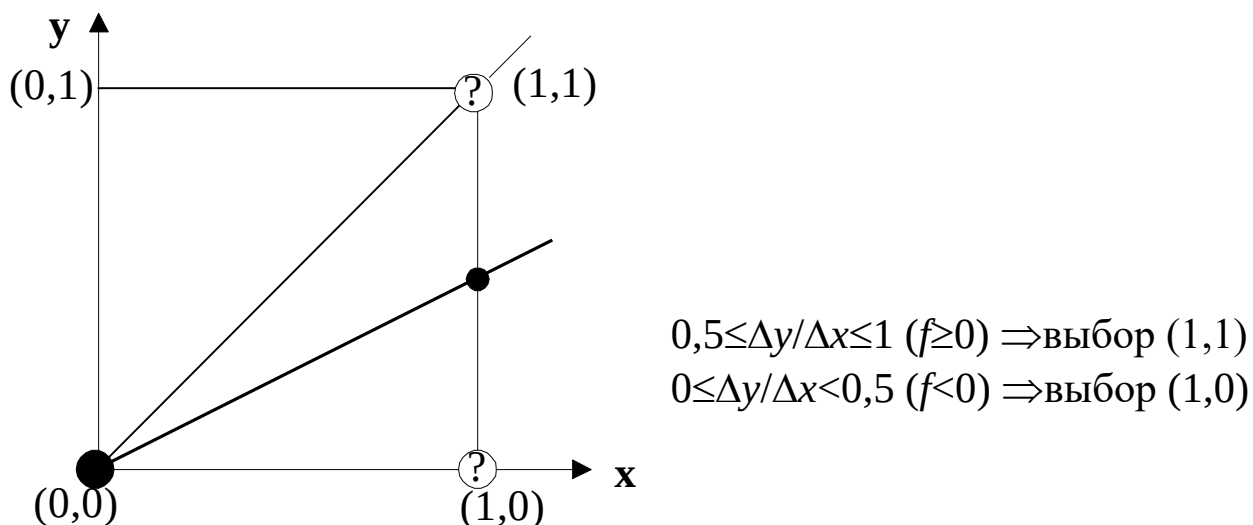


Рис. 2. Основная идея алгоритма Брезенхема

Пусть f_i – значение ошибки на i -м шаге, Y_{ui} – ордината идеального отрезка, Y_i – ордината пикселя, выбранного для аппроксимации отрезка на том же i -м шаге, $\Delta y/\Delta x$ – тангенс угла наклона отрезка. Поскольку на первом шаге высвечивается пиксель с начальными координатами, то для него $f=0$, поэтому задаваемое предварительно значение $f=\Delta y/\Delta x-0,5$ является фактически ошибкой для следующего шага. Ошибка на очередном шаге вычисляется как

$$f_{i+1} = Y_{ui+1} - Y_{i+1} = Y_{ui} + \Delta y/\Delta x - Y_{i+1} = f_i + \Delta y/\Delta x \text{ (если } Y_{i+1} = Y_i).$$

(Поскольку в алгоритме и программе не надо сохранять значения ошибок для всех шагов, то последнее выражение можно записать как $f = f + \Delta y/\Delta x$).

В зависимости от полученного значения ошибки выбирается пиксель с той же ординатой ($Y_{i+1} = Y_i$ при $f < 0$) или пиксель с ординатой, на единицу большей, чем у предыдущего пикселя ($Y_{i+1} = Y_i + 1$ при $f \geq 0$).

Поскольку предварительное значение ошибки вычисляется заранее, то есть $f + \Delta y/\Delta x$ вычислено на предыдущем шаге, то во втором случае останется только вычесть единицу из значения ошибки:

$$f = f - 1.$$

Алгоритм Брезенхема, работающий с действительными величинами, можно записать в следующем виде.

1. Ввод исходных данных: x_1, y_1, x_2, y_2 .

2. Проверка вырожденности отрезка. Если отрезок вырожденный, то выводится точка и осуществляется переход к п.10.
3. Вычисление приращений: $dx:=x_2-x_1$ и $dy:=y_2-y_1$.
4. Вычисление шага изменения каждой координаты пикселя: $sx:=\text{sign}(dx)$, $sy:=\text{sign}(dy)$.
5. Вычисление модулей приращения координат: $dx:=|dx|$, $dy:=|dy|$.
6. Обмен значений dx и dy в зависимости от углового коэффициента наклона отрезка:
 - 1) если $dy > dx$, то выполнить $temp:=dx$, $dx:=dy$, $dy:=temp$, $flag:=1$;
 - 2) иначе $flag:=0$ ($flag$ – флаг, определяющий факт обмена местами координат).
7. Инициализация начального значения ошибки: $f:=dy/dx-0.5$.
8. Инициализация начальных значений координат текущего пикселя: $x:=x_1$, $y:=y_1$.
9. Цикл от 0 до dx делать:
 - 1) вывод точки с координатами (x,y) ;
 - 2) вычисление координат и корректировка ошибки для следующего пикселя:
 - если $f \geq 0$, то если $flag=1$, то $x:=x+sx$, иначе $y:=y+sy$;
 - корректировка ошибки $f:=f-1$;
 - 3) переход к следующему пикселю: если $flag=1$, то $y:=y+sy$ иначе $x:=x+sx$;
 - 4) вычисление ошибки $f:=f+dy/dx$.
10. Конец.

Целочисленный алгоритм Брезенхема

Алгоритм Брезенхема в том виде, как он представлен выше, требует использования арифметики с плавающей точкой и деления (для вычисления углового коэффициента и оценки ошибки). Быстродействие алгоритма можно увеличить, если использовать только целочисленную арифметику и исключить деление. Для этого выражение для вычисления ошибки запишем в виде $f=\Delta y/\Delta x-0,5$ и, умножив обе части этого равенства на $2\Delta x$, получим $2\Delta x f=2\Delta y-\Delta x$. Обозначив $f_u=2\Delta x f$, окончательно получим $f_u=2\Delta y-\Delta x$. В соответствии с этим выражением начальное значение ошибки в п.7 алгоритма будет вычисляться в виде $f:=2*dy-dx$.

Тогда в п.9 алгоритма корректировка ошибки будет производиться по формуле: $f:=f-2*dx$, а вычисление ошибки — $f:=f+2*dy$.

Оптимизация алгоритма Брезенхема – прорисовка линии отрезками переменной длины.

Идея алгоритма Брезенхема – это движение вдоль основной оси по одному пикселю и поддержание текущего отклонения от идеальной прямой в заданных пределах. Если отклонение превышает норму, то делается шаг по неосновной оси, чтобы уменьшить отклонение. Главный момент здесь – это то, что вычисление и проверка производятся на каждом шаге.

Для контроля размещения точек необходимо периодически принимать решения относительно их расстановки, и делать это желательно как можно быстрее. Можно принимать решения для каждой точки как в стандартном алгоритме Брезенхема, но большинство проверок в этом случае излишни.

Идея оптимизации (алгоритм с переменной длиной отрезков) – это движение с каждой итерацией по неосновной оси и проверка текущего отклонения горизонтальной координаты.

Посмотрим на задачу так: когда программа проводит линию в 35 пикселей по X и 10 пикселей по Y, то уже есть много информации, часть которой игнорируется стандартным алгоритмом Брезенхема. Например, поскольку наклон линии находится в пределах 1/3-1/4, то уже можно определённо сказать, что составляющие линию ряды точек состоять из 3 или 4 пикселей (рис. 3). Вычислений становится на 70% меньше.

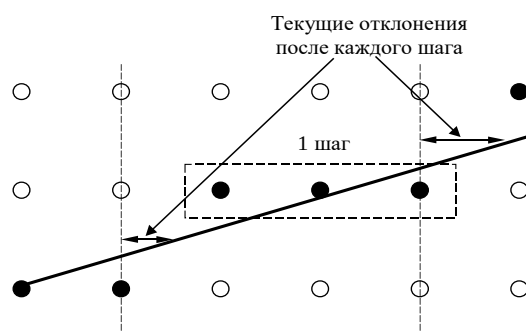


Рис. 3 – Алгоритм с переменной длиной отрезков

Задание

1. Разработать программу, реализующую алгоритмы растеризации отрезков с их последующей прорисовкой. Вывести рисунок в соответствии с вариантом. Для вывода отрезков использовать:

- алгоритм ЦДА;
- алгоритм Брезенхема;
- целочисленный алгоритм Брезенхема;
- встроенные средства языка программирования.

2. Проиллюстрировать разницу между результатами или её отсутствием.

3. Ответить на контрольные вопросы.

4. Оформить отчёт.

Содержание отчёта

1. Титульный лист.
2. Задание.
3. Схемы алгоритмов.
4. Программа на алгоритмическом языке программирования.
5. Пример работы программы.
6. Ответы на контрольные вопросы.

Порядок выполнения работы

1. Ознакомление с целью работы, заданием, содержанием отчёта, порядком выполнения работы, индивидуальным вариантом, примером выполнения работы, примером оформления отчёта.

2. Ознакомление со средствами системы программирования для чтения, обработки и записи графики на примере и с помощью технической документации.

3. Ознакомление с теоретическими материалами по теме.

4. Разработка алгоритма в соответствии с вариантом задания. Алгоритм должен предусматривать:

- 1) чтение файла изображения;
- 2) вывод отрезков в соответствии с заданием по:
 - алгоритму ЦДА;
 - алгоритму Брезенхема;

- целочисленному алгоритму Брезенхема;
- встроенными средствами языка программирования;
- 3) запись полученного изображения в файл;
- 4) иллюстрацию разницы между результатами или её отсутствие.
- 5. Разработка и отладка программы для решения задачи.
- 6. Ответы на контрольные вопросы.
- 7. Оформление отчёта.

Индивидуальные задания

Задание включает в себя вывод фигуры из нескольких отрезков под разными углами.

1. Вывести прямоугольник по заданным левому верхнему и правому нижнему углам.
2. Вывести пентаграмму по заданному радиусу описанной окружности и центральному углу одной из вершин.
3. Вывести звезду Давида по заданному радиусу описанной окружности и центральному углу одной из вершин.
4. Вывести ромб по заданным координатам одной диагонали и длине другой диагонали.
5. Вывести пачифик по заданному радиусу описанной окружности и координатам начала центрального отрезка.
6. Вывести полилинию по заданным координатам четырёх вершин.
7. Вывести параллелограмм по заданным координатам большой диагонали, высоте и углу.
8. Вывести трапецию по заданным координатам трёх вершин и длине средней линии.
9. Вывести правильный семиугольник по заданному радиусу описанной окружности.
10. Вывести связный звёздчатый восьмиугольник по заданному радиусу описанной окружности.
11. По заданным вершинам вывести треугольник и его высоты.
12. По заданным вершинам вывести треугольник и его биссектрисы.
13. По заданным вершинам вывести треугольник и его медианы.
14. По заданным координатам отрезка, углу и количеству звеньев вывести зигзаг.

15. По заданным вершинам вывести треугольник и его срединный треугольник.
16. По двум длинам сторон нарисовать спидрон.
17. По заданным координатам вершин бедра вывести золотой треугольник.
18. По заданным вершинам вывести треугольник и его ортотреугольник.
19. По заданным вершинам вывести треугольник и его подёрный треугольник.
20. По заданным вершинам вывести треугольник и треугольник, построенный по основаниям его биссектрис.

Контрольные вопросы

1. Что такое пиксель?
2. Что такое растровое изображение?
3. Для чего нужны алгоритмы растеризации отрезков?
4. Какие ещё алгоритмы растеризации отрезков существуют?

Пример выполнения работы

Взять n точек, равномерно распределённых по окружности радиуса R и соединить отрезком каждую k -ю с $2*k$ -й по модулю n .

Для растеризации отрезков, заданных координатами концов, реализуем алгоритмы ЦДА, Брезенхема и целочисленный Брезенхема в виде отдельных функций, куда будем передавать координаты вершин и холст.

На форме (рис. 4) предусматриваем 2 объекта `Tbutton`: для нового изображения и сохранения результата, диалог `TsavePictureDialog` для цивилизованного сохранения файла, объект `TImage` для хранения и отображения картинок, 2 объекта `Tedit` для ввода радиуса и количества точек, 2 компонента `Tlabel` для подписи `Tedit`.

При нажатии на кнопку «Создать» из компонентов `Tedit` считываются радиус и количество точек, для битмапа `Image1` выполняется очистка, задаётся формат пикселей `pf24bit`, задаются ширина и высота величиной $2 \text{ радиуса} + 10$, чтобы поместилась окружность, обрамлённая пустотой, выполняется заполнение белым фоном.

Далее вычисляются координаты точек на окружности, и разным цветом прорисовываются отрезки алгоритмами ЦДА, Брезенхема и целочисленным Брезенхема и встроенным методом Tcanvas.line().

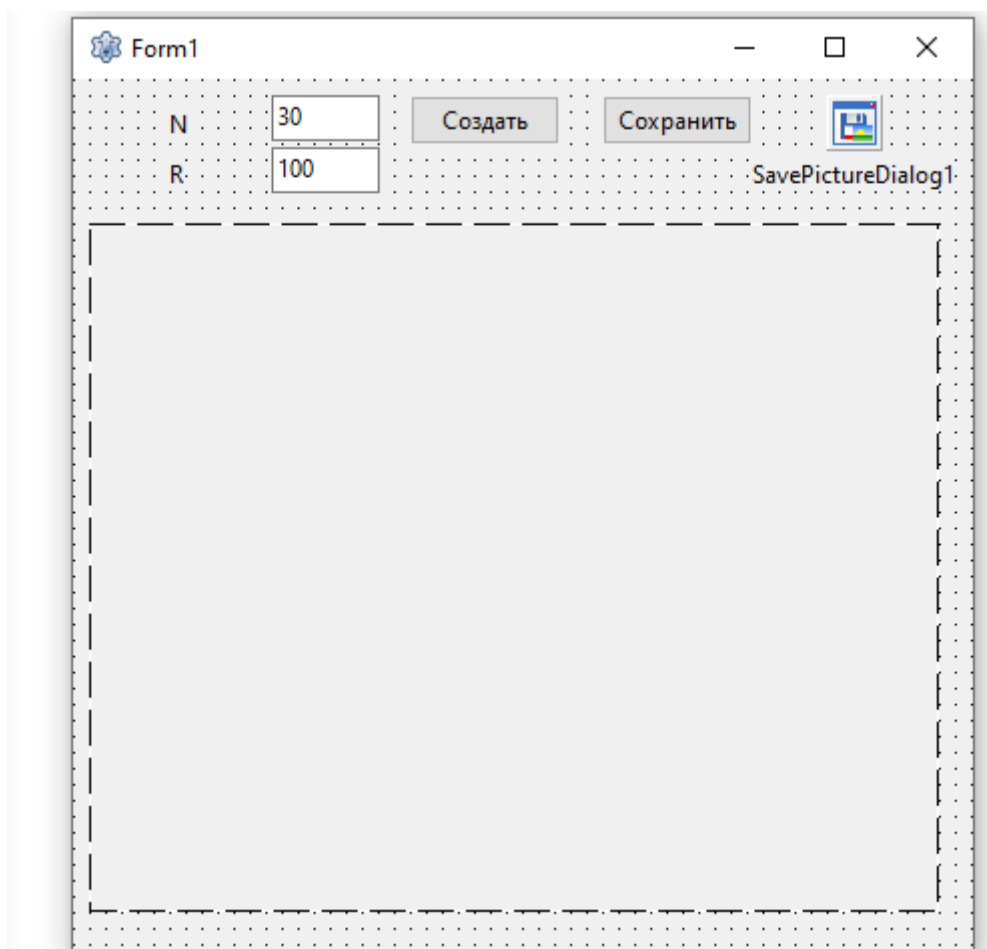


Рис. 4. Форма программы прорисовки отрезков

Получаем модуль приблизительно следующего содержания:

```
unit Unit1;
```

```
{ $mode objfpc } { $H+ }
```

```
interface
```

```
uses
```

```
Classes, SysUtils, FileUtil, Forms, Controls, Graphics, Dialogs,  
StdCtrls, GraphType, IntfGraphics, LCLType, LCLProc, LCLIntf, ExtCtrls,  
ExtDlgs, fpImage, crt, math;
```

type

```
{ TForm1 }
```

```
TForm1 = class(TForm)
```

```
  Button1: TButton;
```

```
  Button3: TButton;
```

```
  Edit6: TEdit;
```

```
  Edit7: TEdit;
```

```
  Label1: TLabel;
```

```
  Label3: TLabel;
```

```
  SavePictureDialog1: TSavePictureDialog;
```

```
  Image1: TImage;
```

```
  procedure Button1Click(Sender: TObject);
```

```
  procedure Button3Click(Sender: TObject);
```

```
private
```

```
  { private declarations }
```

```
public
```

```
  { public declarations }
```

```
end;
```

var

```
  Form1: TForm1;
```

implementation

```
{ $R *.lfm }
```

```
{ TForm1 }
```

```
procedure cda(x1,y1,x2,y2:real; canvas:TCanvas);
```

```
var i: integer;
```

```
  l,dx,dy,x,y:real;
```

```
begin
```

```
  if (x1=x2) and (y1=y2) then
```

```
    begin
```

```
      canvas.Pixels[floor(x1),floor(y1)]:=canvas.Pen.Color;
```

```
    exit
```

```

        end;
    if abs(x2-x1)>=abs(y2-y1) then l:=abs(x2-x1) else l:=abs(y2-y1);
    dx:=(x2-x1)/l;
    dy:=(y2-y1)/l;
    x:=x1+0.5*sign(dx);
    y:=y1+0.5*sign(dy);
    for i:=1 to round(l)+1 do
        begin
            canvas.Pixels[floor(x),floor(y)]:=canvas.Pen.Color;
            x:=x+dx;
            y:=y+dy;
        end;
    end;
end;

```

```

procedure brezf(x1,y1,x2,y2:real; canvas:TCanvas);
var i,sx,sy,f: integer;
    e,dx,dy,x,y,t:real;
begin
    if (x1=x2) and (y1=y2) then
        begin
            canvas.Pixels[floor(x1),floor(y1)]:=canvas.Pen.Color;
            exit
        end;
    sx:=sign(x2-x1);
    sy:=sign(y2-y1);
    dx:=abs(x2-x1);
    dy:=abs(y2-y1);
    x:=x1;
    y:=y1;
    f:=0;
    if dy>dx then
        begin
            t:=dx;
            dx:=dy;
            dy:=t;
            f:=1
        end;
    e:=dy/dx-0.5;

```

```

for i:=0 to round(dx) do
  begin
    canvas.Pixels[floor(x),floor(y)]:=canvas.Pen.Color;
    if e>=0 then
      begin
        if f=1 then x:=x+sx else y:=y+sy;
        e:=e-1;
      end;
    if f=1 then y:=y+sy else x:=x+sx;
    e:=e+dy/dx
  end;
end;

```

```

procedure brezi(x1,y1,x2,y2:integer; canvas:TCanvas);
var t,i,sx,sy,f,dx,dy,x,y,e: integer;
begin
  if (x1=x2) and (y1=y2) then
    begin
      canvas.Pixels[x1,y1]:=canvas.Pen.Color;
      exit
    end;
  sx:=sign(x2-x1);
  sy:=sign(y2-y1);
  dx:=abs(x2-x1);
  dy:=abs(y2-y1);
  x:=x1;
  y:=y1;
  f:=0;
  if dy>dx then
    begin
      t:=dx;
      dx:=dy;
      dy:=t;
      f:=1
    end;
  e:=2*dy-dx;
  for i:=0 to dx do
    begin

```

```

        canvas.Pixels[x,y]:=canvas.Pen.Color;
    if e>=0 then
        begin
            if f=1 then x:=x+sx else y:=y+sy;
            e:=e-2*dx;
        end;
        if f=1 then y:=y+sy else x:=x+sx;
        e:=e+2*dy
    end;
end;

procedure TForm1.Button1Click(Sender: TObject); //Кнопка Создать
var i,n,r:integer;
    alph,x1,x2,y1,y2:real;
begin // С битмапом расположенного на форме TImage
    with Image1.Picture.Bitmap do
        begin
            Clear; //а вдруг там мусор
            PixelFormat:=pf24bit; //формат пикселей
            r:=StrToInt(Edit7.Text);
            Width:=r*2 + 10; //Ширина
            Height:=r*2 + 10; //Высота
            n:=StrToInt(Edit6.Text);
            Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder); //Белый фон
            alph:=2*3.1415926/n;
            for i:=1 to n do
                begin //рисуем отрезками
                    x1:=height div 2 + r*cos(i*alph);
                    y1:=width div 2 + r*sin(i*alph);
                    x2:=height div 2 + r*cos((i*2 mod n)*alph);
                    y2:=width div 2 + r*sin((i*2 mod n)*alph);
                    canvas.Pen.Color:=RGB(127,127,255); //цвет точек
                    cda(x1,y1,x2,y2,canvas);
                    canvas.Pen.Color:=RGB(255,127,127); //цвет точек
                    brezf(x1,y1,x2,y2,canvas);
                    canvas.Pen.Color:=RGB(127,255,127); //цвет точек
                    brezi(round(x1),round(y1),round(x2),round(y2),canvas);
                    canvas.Pen.Color:=RGB(127,127,127); //цвет точек
                end;
            end;
        end;
    end;
end;

```

```

        canvas.line(round(x1),round(y1),round(x2),round(y2));
    end;
end;
end;

procedure TForm1.Button3Click(Sender: TObject);
begin
    if SavePictureDialog1.Execute then
        Image1.Picture.SaveToFile(SavePictureDialog1.FileName);
    end;

end.

```

Смотрим на результат (рис. 5). Можем сохранить.

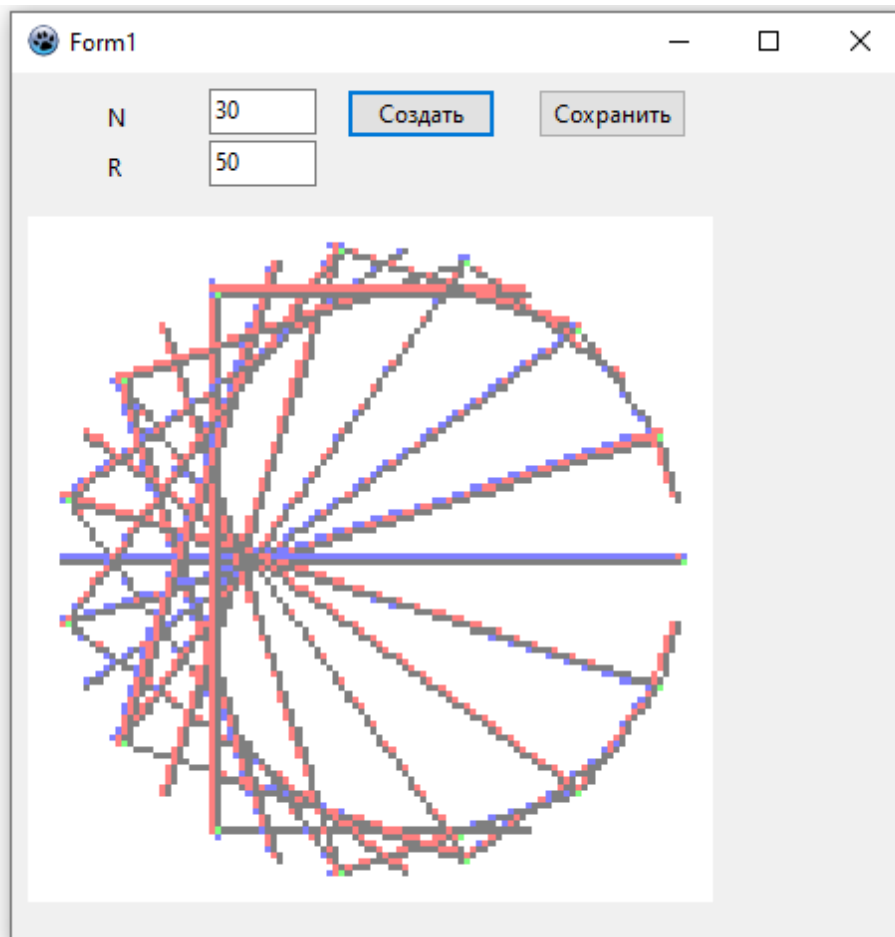


Рис. 5. Результат работы программы прорисовки отрезков